Design and Implementation of a Custom 4-Bit CPU in Logisim

Integrating a 4-Bit ALU, Local Register File, $2^n \times 4$ RAM, and Custom ISA

Computer Architecture & Digital Design Laboratory

Project Overview: In previous laboratory sessions, you designed and verified standalone digital building blocks—including a Random Access Memory (RAM) unit, an Arithmetic Logic Unit (ALU), multiplexers, and registers—in Logisim. In this capstone design assignment, you will integrate those modules to architect, wire, and simulate a complete, programmable **4-Bit Central Processing Unit (CPU)** in Logisim.

1 Core Architectural Specifications

Your CPU must operate as a synchronous sequential circuit driven by a master clock, capable of fetching instructions, decoding control signals, reading and writing operands across local registers and main memory, and executing arithmetic and bitwise logic operations.

1.1 4-Bit Arithmetic Logic Unit (ALU)

Your integrated 4-bit ALU must accept two 4-bit operands ($A[3:0]$ and $B[3:0]$) and an operation select signal (ALU_Op) to perform at least the following six baseline operations:

- **Arithmetic Operations**:
  1. **Addition (`ADD`)**: Computes $S = A + B$ with Carry-Out (C_{out}) and signed Overflow (V) detection.
  2. **Subtraction (`SUB`)**: Computes $D = A - B$ using 2's complement addition ($A + \overline{B} + 1$) with Borrow/Sign detection.
  3. **Multiplication (`MUL`)**: Computes the product $P = A \times B$. Because multiplying two 4-bit numbers produces up to an **8-bit product** ($P[7:0]$), your architecture must explicitly handle the upper and lower 4-bit nibbles (e.g., latching the 8-bit result across two 4-bit local registers such as `HI` and `LO`, or storing the lower 4 bits in a destination register and setting an overflow flag).
- **Bitwise Logic Operations**:
  1. **Bitwise AND (`AND`)**: Computes $Y_i = A_i \cdot B_i$ for $i \in \{0, 1, 2, 3\}$.
  2. **Bitwise OR (`OR`)**: Computes $Y_i = A_i + B_i$ for $i \in \{0, 1, 2, 3\}$.
  3. **Bitwise XOR (`XOR`)**: Computes $Y_i = A_i \oplus B_i$ for $i \in \{0, 1, 2, 3\}$.

1.2 Local CPU Registers (Internal Scratchpad & Special Purpose)

Processors do not read and write directly to main RAM on every ALU cycle; instead, they include fast **local registers** inside the CPU to hold intermediate operands, temporary computation results, and execution state. Your CPU must include:

- **General-Purpose / Local Data Registers**: Design a local register file containing **2 to 4 general-purpose 4-bit registers** (e.g., R_0, R_1, R_2, R_3, or an Accumulator `ACC` paired with auxiliary local registers R_1, R_2, R_3). These registers must store local intermediate values during multi-step expressions—such as computing $(X+Y) \times Z$ or $(A \oplus B) - C$ locally—before writing the final result back to RAM.
- **Extended Result Register(s)**: Include either a dedicated 4-bit high-nibble register (`HI` / `REM`) or pair two local registers ($R_{\text{high}} : R_{\text{low}}$) to hold the upper 4 bits of an 8-bit multiplication product or the 4-bit remainder of a division operation.
- **Special-Purpose Control Registers**:
  - **Program Counter (`PC`)**: Holds the address of the next instruction to fetch.
  - **Instruction Register (`IR`)**: Latches the current instruction word being decoded and executed.
  - **Status / Flags Register (`FLAGS`)**: Latches condition bits from the ALU, including Zero (Z), Carry (C), Negative/Sign (N), and Overflow (V).

1.3 $2^n \times 4$-Bit Data RAM Configuration

Integrate a $2^n \times 4$-bit RAM module to store program data, input arrays, and final output values:

- **Choosing Address Width** n: Select and justify an appropriate address width n (for example, $n = 4$ for a 16×4-bit RAM, $n = 6$ for a 64×4-bit RAM, or $n = 8$ for a 256×4-bit RAM) so that all test operands, intermediate arrays, and outputs fit comfortably.
- **Memory Interface**: Provide data transfer instructions (`LOAD` and `STORE`) that move 4-bit nibbles between any address in your $2^n \times 4$ RAM and your local CPU registers.
- **Memory Organization Choice**: You may implement either a **Harvard architecture** (separate Instruction ROM/RAM of width W bits paired with a $2^n \times 4$-bit Data RAM) or a **Von Neumann architecture** (shared $2^n \times 4$ RAM where multi-nibble instructions are fetched across consecutive clock cycles into the `IR`). Clearly state and justify your choice in your report.

2 Detailed Instruction Set Architecture (ISA) Design Guide

A major component of this project is designing your own **Instruction Set Architecture (ISA)**. Rather than following a pre-assigned bit layout, you must calculate how many bits each instruction requires and define how its fields are encoded.

2.1 Step 1: Calculate How Many Bits Are Needed for an Instruction

The total bit-width W of an instruction word is determined by three architectural parameters:

$$W = N_{\text{opcode}} + N_{\text{reg_fields}} + N_{\text{address / immediate}} \tag{1}$$

1. **Opcode Field Width (N_{opcode})**: Count the total number of distinct instructions M your CPU will execute:
   - 6 baseline ALU operations (`ADD`, `SUB`, `MUL`, `AND`, `OR`, `XOR`)
   - 3–4 data movement operations (`LOAD` from RAM, `STORE` to RAM, `LDI` load 4-bit immediate constant, `MOV` register-to-register copy)
   - 2–6 custom extension and control instructions (e.g., bit-shift, division, conditional branch, `HALT`)

The minimum number of opcode bits required to uniquely encode M instructions is:

$$N_{\text{opcode}} = \lceil \log_2(M) \rceil \tag{2}$$

For example, supporting between 9 and 16 instructions requires $N_{\text{opcode}} = \lceil \log_2(16) \rceil = \mathbf{4}$ bits (giving opcodes `0000` through `1111`).

2. **Local Register Specifier Width (N_{reg})**: If your CPU contains K local general-purpose registers, selecting a single register requires:

$$N_{\text{reg}} = \lceil \log_2(K) \rceil \text{ bits per register operand} \tag{3}$$

For $K = 4$ local registers (R_0, R_1, R_2, R_3), each register identifier requires **2** bits. A two-operand register instruction specifying both a destination register (R_d) and a source register (R_s) uses $2 + 2 = 4$ bits for register selection.

3. **RAM Address or Immediate Field Width (n or 4 bits)**:
   - Loading a 4-bit immediate constant (`Imm4`) directly into a local register requires a **4-bit immediate field**.
   - Directly addressing a location in your $2^n \times 4$ RAM requires an **n-bit address field** (or, if $n > 4$ in a compact instruction format, you may use register-indirect addressing where a local register or register pair holds the RAM address).

2.2 Step 2: Structure Your Instruction Word Formats

Based on Equation (1), you may choose an instruction word width W (typically **8-bit**, **10-bit**, or **12-bit**) that cleanly accommodates your register and memory fields. Table 1 outlines two standard reference patterns you can adapt:

Design Option	**Bit Allocation**	**How Instructions Are Encoded**
Option A: 8-Bit ISA	`[7:4]` Opcode (4b)	**R-Type (ALU)**: `[Opcode:4b] [Rd:2b] [Rs:2b]` executes local register operation $R_d \leftarrow R_d$ op R_s.
($W = 8$ bits, $K = 4$ regs)	`[3:0]` Operands (4b)	**M/I-Type (Memory/Imm)**: Uses an implicit target register (such as R_0/`ACC`) or register-indirect addressing so `[3:0]` holds a 4-bit RAM address ($n = 4$) or 4-bit immediate constant.
Option B: 10/12-Bit ISA	`[9:6]` Opcode (4b)	**R-Type (ALU)**: `[Opcode:4b] [Rd:2b] [Rs1:2b] [Rs2:2b]` (3-register format $R_d \leftarrow R_{s1}$ op R_{s2}) or 2-register format with unused trailing bits.
($W = 10$–12 bits, $K = 4$ regs)	`[5:4]` Reg R_d (2b) `[3:0]` Addr/Imm (nb)	**M/I-Type (Memory/Imm)**: `[Opcode:4b] [Rd:2b] [Addr/Imm:4-6b]` transfers data between any local register R_d and any $2^n \times 4$ RAM address in a single instruction.

Table 1: **Representative Instruction Encoding Strategies.** You must define and justify your exact bit fields, opcode mappings, and addressing modes in your design report.

2.3 Step 3: Required ISA Documentation Table

In your submission report, provide a complete ISA specification table showing the binary opcode, mnemonic, instruction syntax, and Register Transfer Level (RTL) behavior for every instruction. Table 2 shows the expected structure:

Opcode	Mnemonic	Syntax	RTL Description (Local Registers & $2^n \times 4$ RAM)
0000	LOAD	LOAD Rd, [Addr]	$R_d \leftarrow$ RAM[Addr] (Loads 4-bit value from RAM into local register R_d)
0001	STORE	STORE Rd, [Addr]	RAM[Addr] $\leftarrow R_d$ (Stores 4-bit value from local register R_d into RAM)
0010	LDI	LDI Rd, Imm4	$R_d \leftarrow$ Imm4 (Loads 4-bit constant directly into local register R_d)
0011	ADD	ADD Rd, Rs	$R_d \leftarrow R_d + R_s$; updates Carry (C), Overflow (V), and Zero (Z) flags
0100	SUB	SUB Rd, Rs	$R_d \leftarrow R_d - R_s$; updates Sign (N), Borrow, and Zero (Z) flags
0101	MUL	MUL Rd, Rs	HI : $R_d \leftarrow R_d \times R_s$ (8-bit product split across HI and R_d)
0110	AND	AND Rd, Rs	$R_d \leftarrow R_d \wedge R_s$; updates Zero ($Z$) flag
0111	OR	OR Rd, Rs	$R_d \leftarrow R_d \vee R_s$; updates Zero ($Z$) flag
1000	XOR	XOR Rd, Rs	$R_d \leftarrow R_d \oplus R_s$; updates Zero ($Z$) flag
1001-1110	*Custom*	*Student-Defined*	*Reserved for your open-ended special instructions and hardware extensions*
1111	HALT	HALT	Stops Program Counter increment and freezes CPU state for inspection

Table 2: **Sample ISA Specification Template.** Adapt this table to reflect your chosen instruction bit-width, local register architecture, and custom opcodes.

3 Open-Ended Architectural Enhancements (Innovation Task)

Beyond implementing the six baseline ALU operations, local register file, and $2^n \times 4$ RAM interface, you are required to **think independently and engineer custom functional extensions** into your CPU. This component is intentionally open-ended so you can explore hardware optimizations and specialized functional units.

How to Approach the Open-Ended Requirement: Identify a computational bottleneck, arithmetic limitation, or control-flow need in a baseline 4-bit processor, and design dedicated digital logic in Logisim to address it. Assign opcode(s) to your custom feature(s) in your ISA and demonstrate the hardware working in simulation.

Examples of strong architectural enhancements include (you may select from these or invent your own):

- **Power-of-2 Shift Optimization**: Dedicated shifter hardware or instructions (SHL/SHR) that execute multiplication or division when an operand is a power of 2 ($2^1, 2^2, 2^3$) via single-cycle bit shifting rather than full array multiplication/division.
- **Hardware Integer Division Unit (DIV)**: A combinational or sequential divider circuit that divides two 4-bit numbers and stores both the **4-bit Quotient** and **4-bit Remainder** into separate local registers (with divide-by-zero flag detection).
- **Conditional Branching & Looping (JMP, JZ, JC)**: Branch decision logic connected to the FLAGS register and Program Counter multiplexer, enabling loops (such as summing an array of nibbles stored in the $2^n \times 4$ RAM).
- **Multi-Nibble / 8-Bit Chained Arithmetic (ADC/SBB)**: Using the Carry/Borrow flag to chain two 4-bit additions or subtractions across local register pairs to compute 8-bit results.

- **Bit Manipulation or Hardware Comparison (`ROL`, `ROR`, `NOT`, `CMP`)**: Bit-rotation units or magnitude comparators that update condition flags without overwriting local register operands.

4 Deliverables and Verification Requirements

1. **Modular Logisim Circuit File (`.circ`)**:
   - Build separate, clearly labeled subcircuits for the **4-Bit ALU**, **Local Register File**, **Control Unit / Instruction Decoder**, and **Top-Level CPU Datapath**.
   - Include visible hex/binary probes on the top-level canvas displaying the current `PC`, `IR` (decoded instruction fields), all local registers ($R_0 \dots R_{K-1}$, `HI`), `FLAGS`, ALU inputs/outputs, and active RAM address/data buses.
2. **Architecture & ISA Design Report**:
   - Provide your mathematical derivation for instruction bit-width W, local register count K, and RAM dimensions $2^n \times 4$.
   - Include your complete Instruction Encoding diagram, Opcode/RTL table, Control Unit truth table, and Datapath block diagram.
   - Explain the internal logic and performance benefit of your open-ended special feature(s).
3. **Self-Contained Verification Program**:
   - Pre-load a test program into your memory image that loads operands from the $2^n \times 4$ RAM into local registers, executes all six baseline operations (`ADD`, `SUB`, `MUL`, `AND`, `OR`, `XOR`), demonstrates local register reuse across multi-step calculations, executes your custom enhancement instruction(s), and writes the verified results back to RAM.